UNIVERSITY OF MINES AND TECHNOLOGY
FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

REPORT ENTITLED

BUILDING A DISTRIDUTED IDS FOR MULTI SEGMENT NETWORK MONITORING

BY
NAME: BOTUO THERESAH

INDEX NUMBER: FCM.41.018.109.23

COURSE: NETWORK SECURITY, AUDITING AND MONITORING (CY376)
BLUE TEAM
CLASS: CY III

SEMESTER: TWO

COURSE INSTRUCTOR

FREDERICK EDEM BRONI (MR)

TARKWA, GHANA

AUGUST 2026

Abstract

Modern enterprise networks are commonly divided into multiple segments — such as a demilitarized zone (DMZ), an internal user network, and a management or security-operations segment — to contain the impact of a compromise in any single zone. However, segmentation alone does not provide visibility into malicious activity occurring within or between these zones. This project addresses that gap by designing, implementing, and evaluating a distributed intrusion detection system (IDS) for a three-segment network. The system deploys a dedicated Suricata sensor within each segment to inspect local traffic in real time, with all sensors forwarding alerts to a centralized Wazuh manager for correlation, storage, and visualization. A pfSense firewall/router provides inter-segment routing and access control, while a Kali Linux host is used to generate representative attack traffic (port scans, vulnerability scans, and flooding attempts) for evaluation. [Summarize your key results here once testing is complete, e.g., number and types of alerts generated, sensor performance, any detection gaps identified.] The findings demonstrate [state your overall conclusion once available] and inform recommendations for extending distributed monitoring in segmented network environments.

Table of Contents

1. Introduction

Network segmentation is a foundational control in modern security architecture: by dividing a network into zones with distinct trust levels — for example, a public-facing DMZ, an internal user LAN, and a management or security-operations segment — an organization limits how far an attacker can move after gaining an initial foothold. Segmentation, however, is a containment control, not a detection control. Without monitoring inside each segment and a way to correlate what is seen across segments, malicious activity can persist undetected for long periods, and analysts lose the ability to reconstruct how an incident unfolded across the network.

This project takes the Blue Team perspective on that problem. Rather than focusing on offensive techniques, the objective is to design and build a monitoring capability that gives a defender visibility into activity across every segment of a small, representative multi-segment network, and to centralize that visibility so it can be acted on from a single point rather than three disconnected logs.

1.1 Problem Statement

A segmented network without per-segment monitoring creates blind spots: traffic that never crosses the router/firewall boundary (for example, lateral movement between two hosts on the same internal segment) is invisible to a single, centrally placed IDS. A genuinely distributed IDS is needed, one sensor per segment, so that intra-segment as well as inter-segment traffic can be inspected.

1.2 Project Objectives

- Design a segmented network topology representative of a small organization (DMZ, internal LAN, management/SOC segment).
- Deploy a dedicated Suricata IDS sensor within each segment to provide local, real-time traffic inspection.
- Centralize alerting and correlation using a Wazuh manager, so that all sensors report to a single dashboard.
- Generate representative attack traffic (reconnaissance, vulnerability scanning, and flooding) to validate detection coverage.

- Evaluate the resulting system against recognized frameworks (MITRE ATT&CK, CIS Benchmarks) and produce recommendations for improvement.

1.3 Scope

The project is implemented entirely within an isolated virtual lab (VMware Workstation) using simulated hosts and traffic. No production systems, real user data, or unauthorized third-party targets are involved at any stage, consistent with the course's academic integrity requirements for lab-based work.

2. Literature and Tooling Review

2.1 Intrusion Detection Approaches

IDS technology is generally split into signature-based and anomaly-based detection. Signature-based systems match traffic against a database of known attack patterns and are effective against well-documented threats but blind to novel ones; anomaly-based systems instead model normal behavior and flag deviations, catching unknown attacks at the cost of a higher false-positive rate. Most production deployments, including the one built for this project, combine both approaches: a signature engine for known threats, supplemented by manager-side correlation rules that can flag unusual patterns across multiple sensors.

2.2 Suricata

Suricata is an open-source, multi-threaded network IDS/IPS engine capable of signature-based detection, protocol identification, and file extraction from live traffic. Its native structured logging format (EVE JSON) makes it straightforward to forward alerts into a external log management and correlation platform, which is the property this project relies on to connect three independent sensors to one central manager. Suricata's multi-threaded design also allows it to keep pace with higher traffic volumes than older single-threaded engines, which matters when a sensor is expected to inspect all traffic within a segment rather than a single link.

2.3 Wazuh

Wazuh is an open-source security monitoring platform that combines a central manager, an indexer for log storage/search, and a dashboard for visualization, alongside lightweight agents installed on monitored hosts. In this project, Wazuh serves as the correlation layer: each Suricata sensor's agent forwards the EVE JSON log to the manager, where alerts from all three segments become visible and searchable from a single dashboard. This centralization is the core

architectural idea being tested — that a distributed set of sensors is only useful to a defender if their output is unified rather than left as three separate logs.

2.4 MITRE ATT&CK

The MITRE ATT&CK framework catalogs adversary tactics and techniques observed in real intrusions and is widely used to structure both offensive testing and defensive coverage assessments. This project maps the attack traffic generated during testing (for example, network scanning and discovery activity) to relevant ATT&CK techniques, giving the results and findings section a standard vocabulary rather than ad-hoc descriptions.

2.5 CIS Benchmarks

The CIS Benchmarks provide vendor-specific, consensus-based hardening guidance for operating systems, network devices, and applications. Where relevant, this project's implementation and analysis sections reference applicable CIS guidance for the router/firewall and host configurations used, to ground configuration choices in a recognized standard rather than personal preference.

2.6 Related Architectural Approach: pfSense

pfSense is an open-source firewall/router distribution built on FreeBSD, offering stateful firewalling, routing between multiple interfaces, and DHCP/DNS services through a web-based interface. It is used here purely as the segmentation and routing backbone connecting the three lab segments; the security monitoring itself is deliberately kept separate, on the dedicated Suricata sensors, so that the router's role stays limited to traffic forwarding and access control rather than detection.

3. Methodology

3.1 Lab Environment

The lab was built entirely in VMware Workstation on a single host machine. All addressing, traffic, and attack activity described in this report is confined to this isolated virtual environment.

3.2 Network Topology

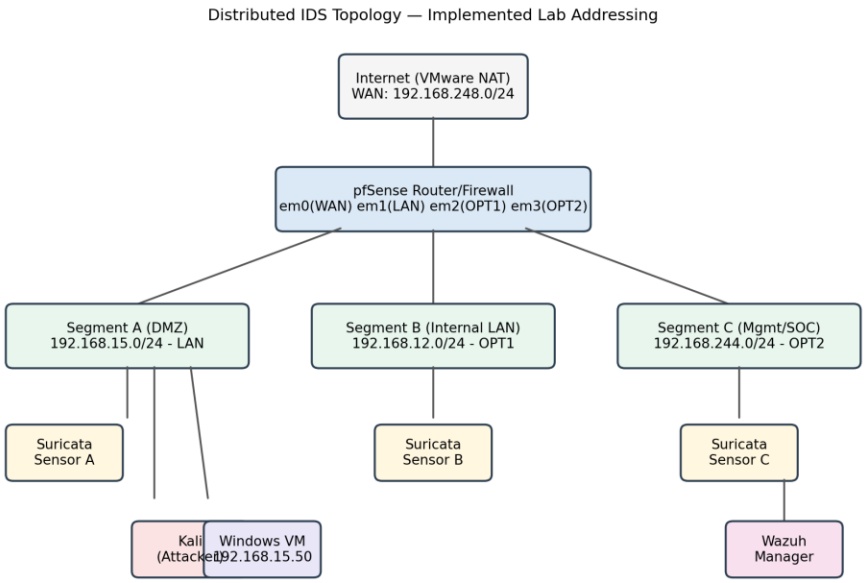

Figure 1: Distributed IDS network topology, showing the addressing actually implemented in the lab

The topology consists of a pfSense router/firewall connecting three isolated segments: Segment A (DMZ, 192.168.15.0/24), Segment B (Internal LAN, 192.168.12.0/24), and Segment C (Management/SOC, 192.168.244.0/24). These subnets were auto-assigned by VMware Workstation when the corresponding host-only virtual networks (VMnet2, VMnet3, VMnet4) were created, and were adopted as-is rather than forced to arbitrary values, since the specific subnet numbers are not architecturally significant. Each segment hosts a dedicated Suricata sensor. The Wazuh manager resides on Segment C, reflecting its role as the security-operations zone. A Kali Linux host on Segment A generates attack traffic representing an external or DMZ-originated threat.

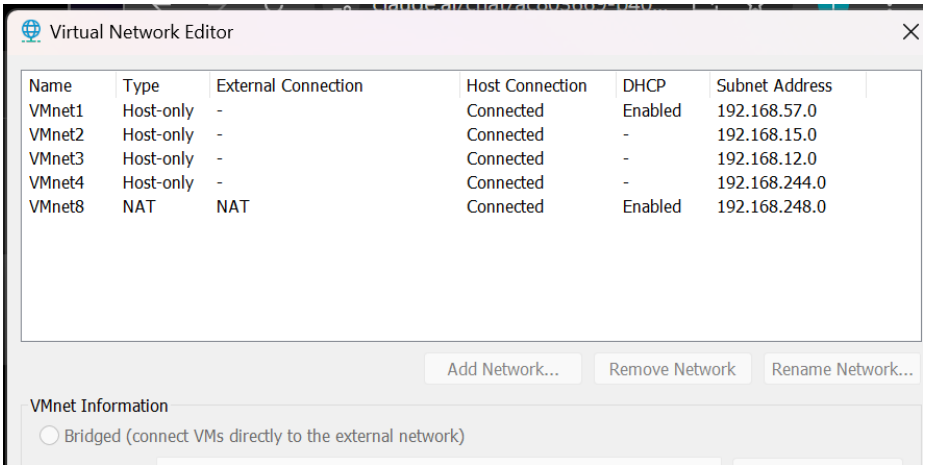

Figure 2: VMware Virtual Network Editor showing the three host-only segments (VMnet2, VMnet3, VMnet4) with DHCP disabled, since pfSense provides DHCP for each segment

3.3 Tools and Justification

Tool	Role in Lab	Justification
pfSense	Router/firewall connecting the three segments	Free, GUI-driven, widely documented; keeps routing and monitoring cleanly separated
Suricata	Per-segment IDS sensor	Multi-threaded, native JSON (EVE) logging integrates directly with Wazuh
Wazuh	Central alert correlation and dashboard	Combines manager, indexer, and dashboard; agent model matches distributed sensor design
Kali Linux	Attack traffic generation	Standard, well-documented toolset (nmap, hping3) for reproducible test cases

3.4 Procedure

The lab was built in the following sequence:

- Configured VMware virtual switches for the three isolated segments and the pfSense NAT/WAN link.
- Installed and configured pfSense, assigning WAN and three LAN interfaces with the addressing scheme above, and enabling DHCP per segment.
- Deployed one Ubuntu Server VM per segment and installed Suricata, setting HOME_NET to the local subnet and enabling EVE JSON logging.
- Deployed the Wazuh manager (indexer, server, dashboard) and registered a Wazuh agent on each Suricata sensor, configured to forward the eve.json log.
- Deployed a Kali Linux VM and generated reconnaissance and flooding traffic against hosts in each segment.
- Captured alerts from Suricata's local logs and the centralized Wazuh dashboard, and mapped observed activity to MITRE ATT&CK techniques.

3.5 Deviations and Issues Encountered

Two notable issues arose during the build and are documented here rather than omitted, since diagnosing them is itself part of the methodology:

- A VMware e1000 virtual NIC / FreeBSD checksum-offload interaction caused intermittent, silent ICMP failures between pfSense and directly connected hosts, despite correct IP addressing and successful ARP resolution. This was resolved by disabling

hardware checksum offload, TCP segmentation offload, and large receive offload under System > Advanced > Networking in pfSense, followed by a full reboot of the affected VMs.

- DNS resolution initially failed on the Segment A sensor (affecting both suricata-update's rule download and general internet reachability), traced to the same underlying offload/driver inconsistency; resolving it required disabling NIC offload features (ethtool) on the Ubuntu host and rebooting rather than restarting networking services alone.

4. Implementation

This section documents the concrete configuration of each component, with evidence captured directly from the lab as it was built.

4.1 pfSense Configuration

pfSense CE 2.8.1-RELEASE was installed as a VM with four network adapters: one on VMware NAT (WAN) and three custom adapters mapped to VMnet2/3/4 (Segments A, B, and C). During first boot, the four detected interfaces (em0-em3) were assigned to WAN, LAN, OPT1, and OPT2 respectively.

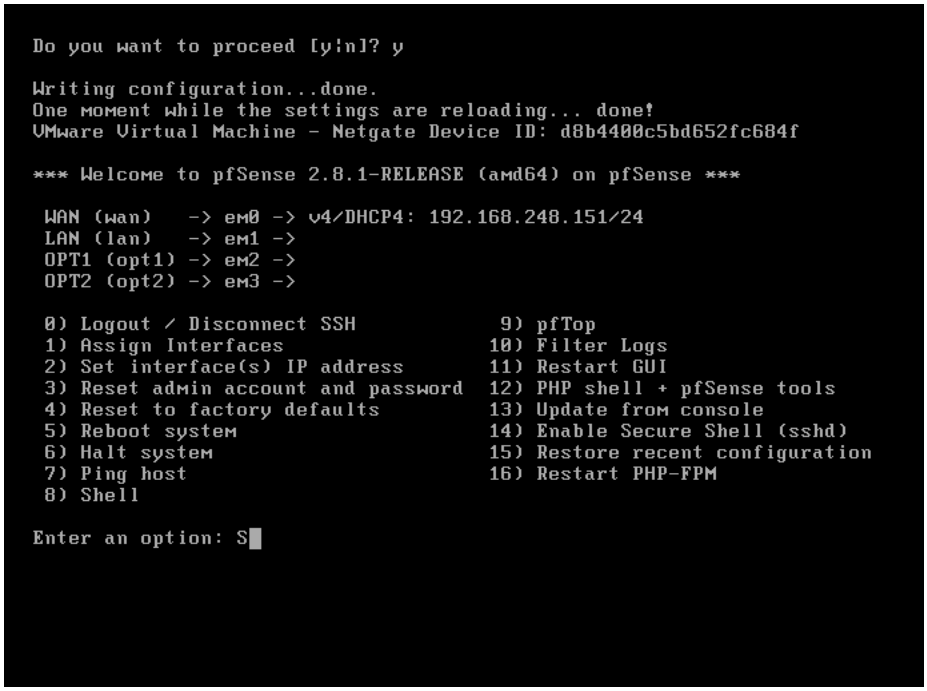

Figure 3: pfSense console showing detected interfaces (em0-em3) prior to assignment

Static IP addressing was then applied via the console: LAN (Segment A) at 192.168.15.1/24, OPT1 (Segment B) at 192.168.12.1/24, and OPT2 (Segment C) at 192.168.244.1/24, with WAN left on DHCP to obtain internet access through VMware's NAT.

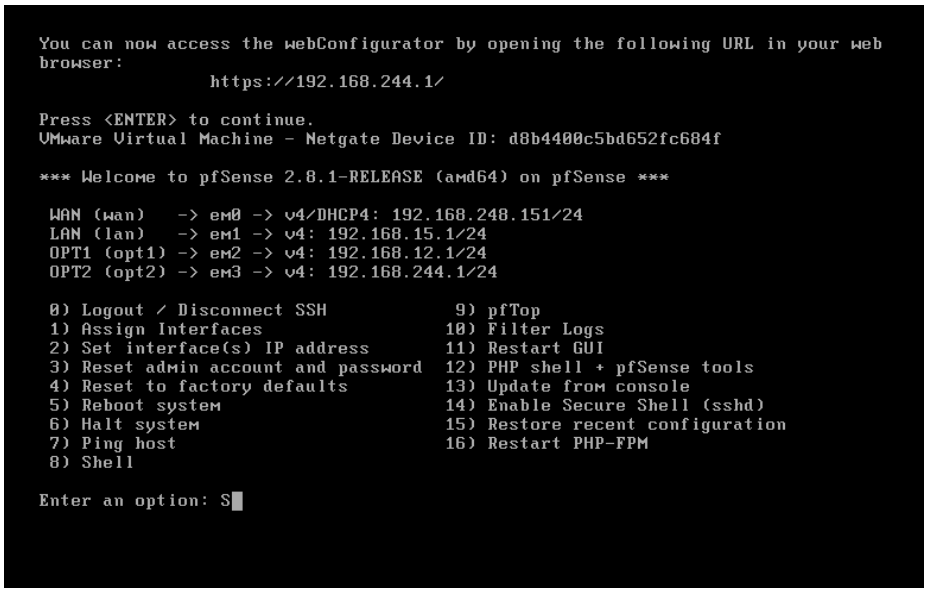

Figure 4: pfSense console summary confirming all four interfaces addressed correctly

OPT1 and OPT2 were then enabled and renamed to SEGMENTB and SEGMENTC respectively through the pfSense web GUI, and DHCP was enabled on all three internal interfaces with a .100-.200 address pool on each subnet.

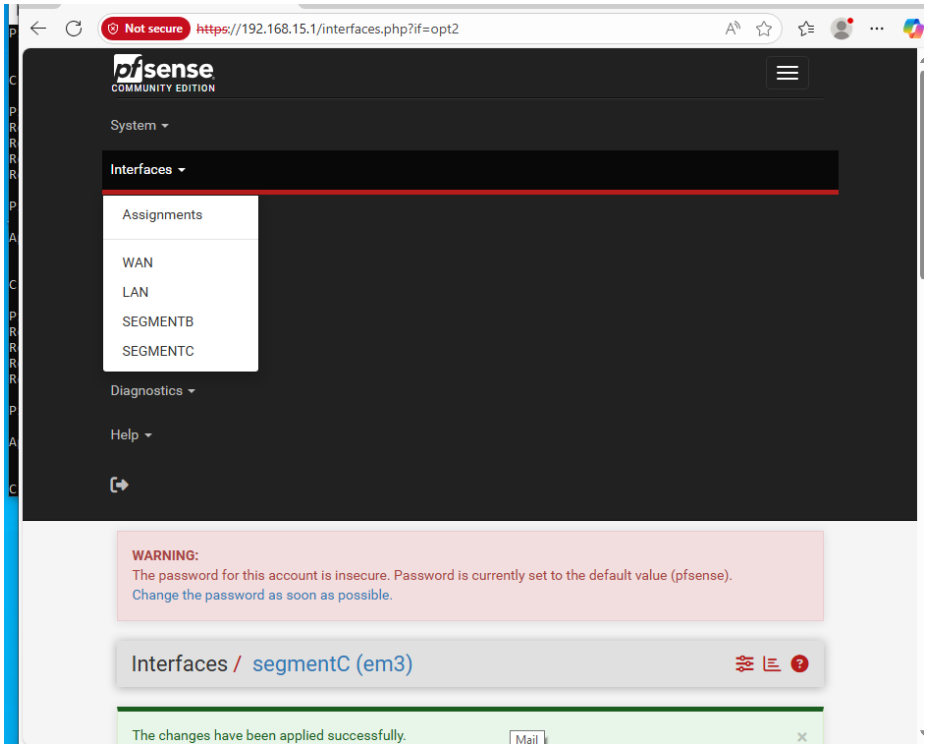

Figure 5: pfSense web GUI Interfaces menu confirming WAN, LAN, SEGMENTB, and SEGMENTC are all configured

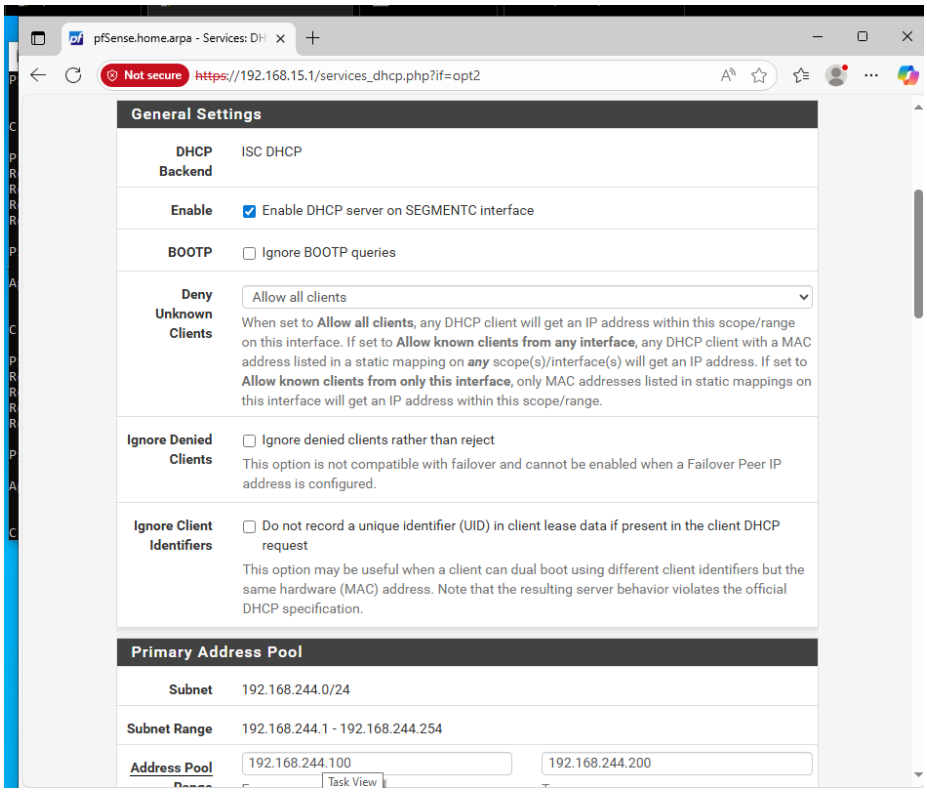

Figure 6: DHCP server configuration for Segment C (192.168.244.0/24), enabled with pool range 192.168.244.100-192.168.244.200

Outbound NAT was left on pfSense's default Automatic mode, and the default 'allow LAN to any' firewall rule (auto-created per interface) was confirmed present, giving each segment outbound connectivity without additional manual rules at this stage of the build.

4.2 Suricata Deployment

Suricata 8.0.6 was installed on an Ubuntu Server 22.04 VM via the OISF stable PPA, then configured with a per-segment HOME_NET and the sensor's actual NIC name. During the first sensor's setup, the default suricata.yaml still referenced the placeholder interface eth0, which does not exist under Ubuntu's predictable network naming; this was identified via a targeted grep and corrected to ens33, the sensor's real interface.

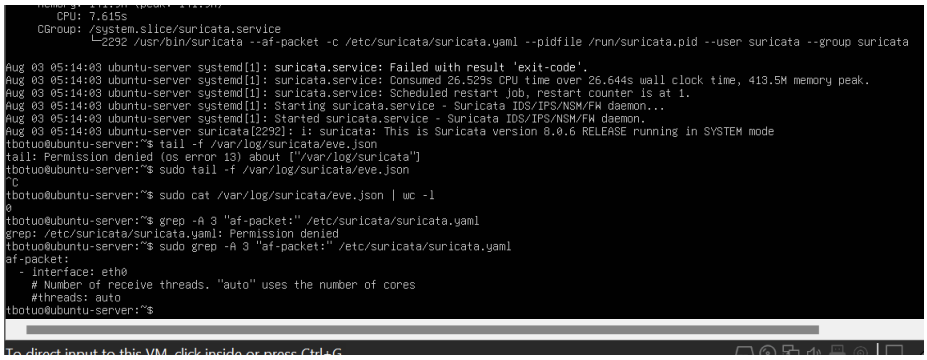

Figure 7: suricata.yaml confirmed to reference the non-existent eth0 interface before correction to ens33

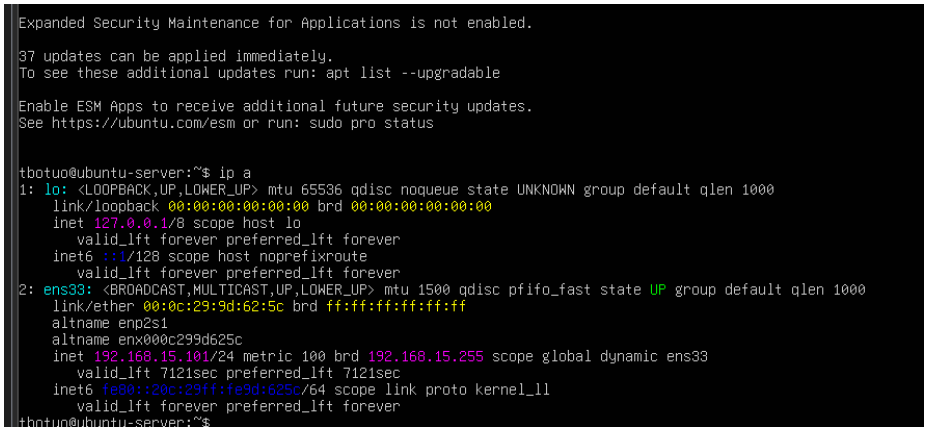

Figure 8: Segment A sensor network configuration (ens33, 192.168.15.101/24) used to set HOME_NET and the af-packet interface

After correcting the interface and updating HOME_NET to each segment's subnet, suricata-update was run to fetch the Emerging Threats Open ruleset (68,097 rules total, 52,158 enabled after default exclusions), and the service was enabled and verified as active.

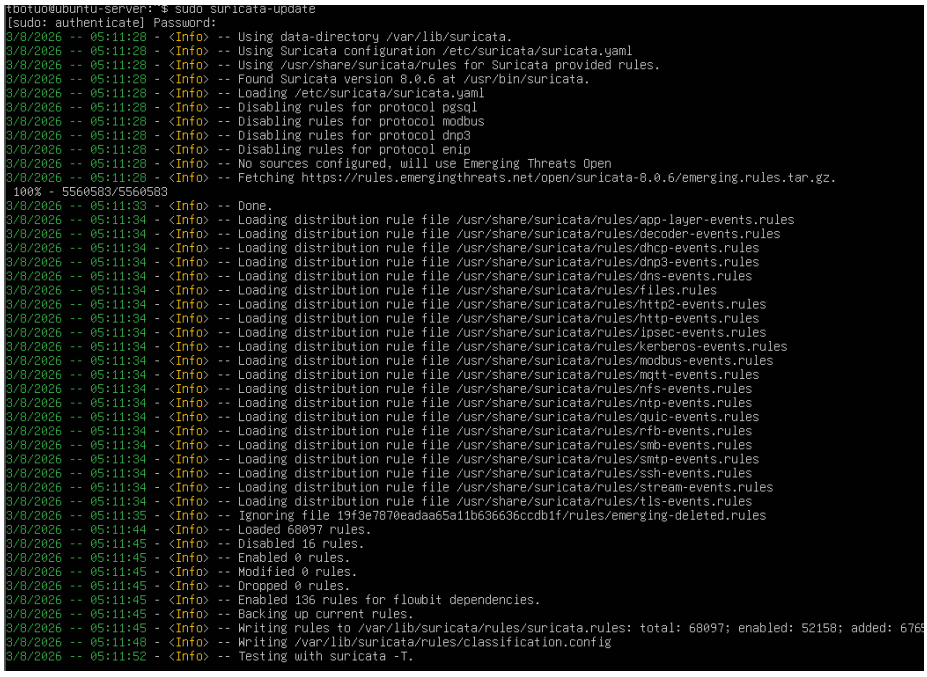

Figure 9: suricata-update output confirming the Emerging Threats Open ruleset was fetched and loaded

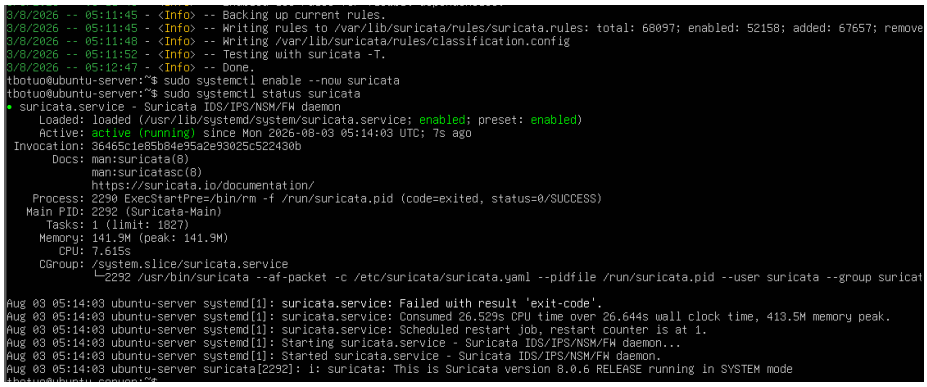

Figure 10: systemctl status confirming the Suricata service active (running) on the Segment A sensor

Correct packet capture was validated by tailing the EVE JSON log while generating ICMP traffic from another host on the same segment; flow-type events referencing the correct source IP and inbound interface confirmed the sensor was genuinely inspecting live segment traffic rather than sitting idle.

Figure 11: eve.json flow event confirming Suricata captured real traffic (src_ip 192.168.15.50) on interface ens33

The validated Segment A sensor VM was then cloned twice (full clones) to produce the Segment B and Segment C sensors, with each clone's network adapter reattached to the corresponding VMnet and HOME_NET updated to that segment's subnet.

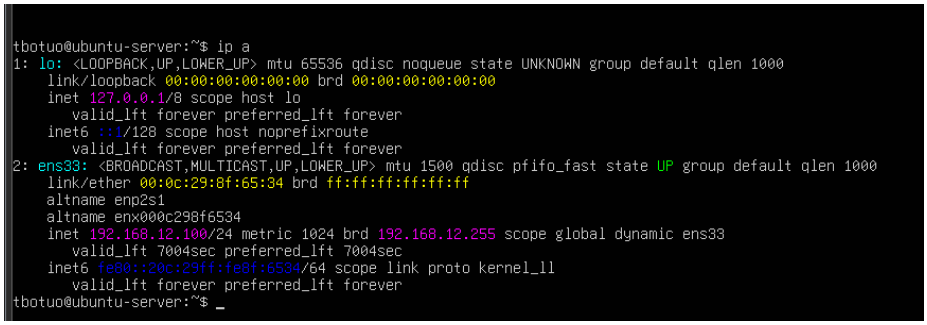

Figure 12: Segment B sensor after cloning, confirmed on 192.168.12.100/24 via DHCP from pfSense's SEGMENTB interface

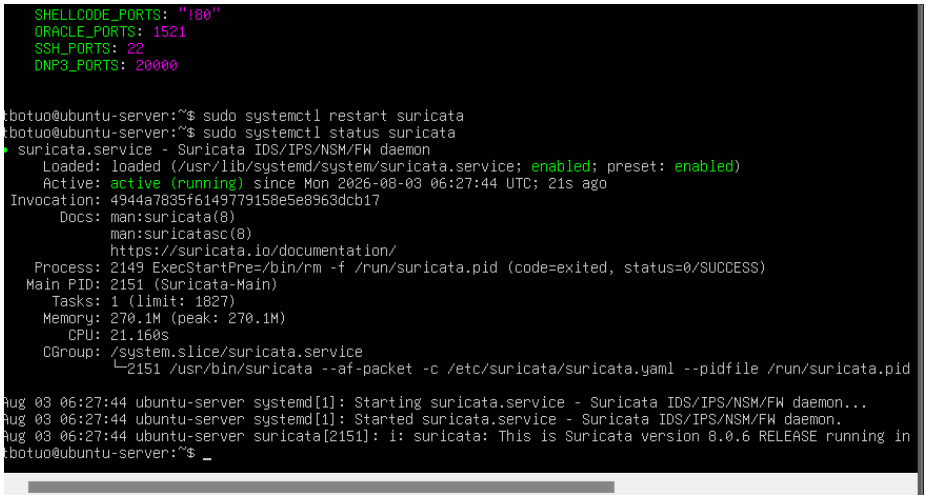

Figure 13: Segment B Suricata service active (running) after HOME_NET was updated to 192.168.12.0/24

The same clone-and-reconfigure process was repeated for the Segment C sensor (HOME_NET set to 192.168.244.0/24), which was confirmed running using the same verification steps.

4.3 Connectivity Troubleshooting (Evidence)

The deviations noted in Section 3.5 are evidenced here. Diagnostics included checking pfSense's ARP table (confirming Layer 2 resolution was intact even while ICMP failed), and reviewing DNS resolver configuration on the affected sensor.

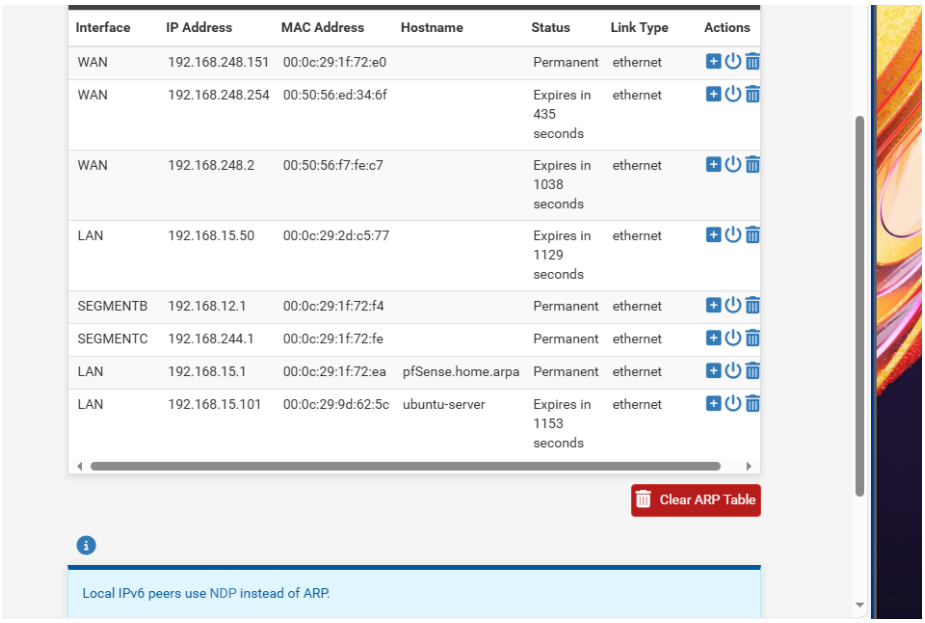

Figure 14: pfSense ARP table showing correct MAC resolution for the Ubuntu sensor despite ICMP failing, pointing to an offload-related issue rather than an addressing or Layer-2 fault

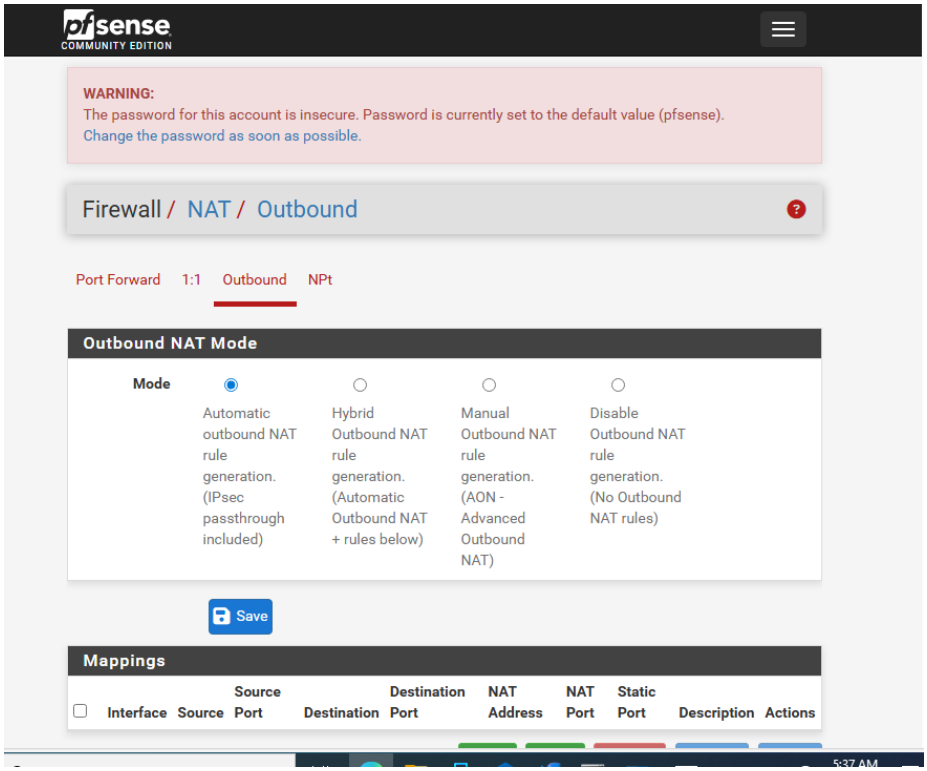

Figure 15: Outbound NAT confirmed on Automatic mode, ruling out a NAT misconfiguration as the cause

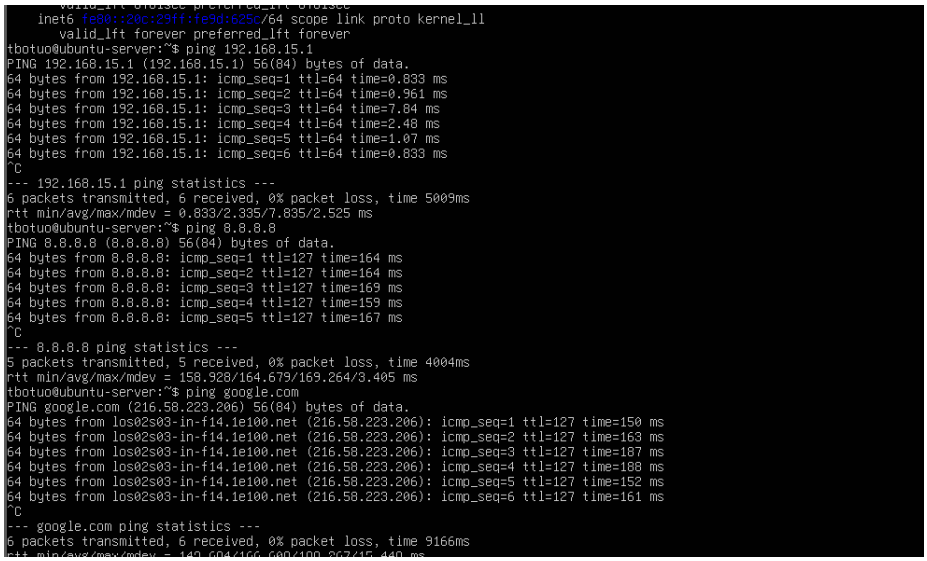

Figure 16: Full connectivity (gateway, public IP, and DNS) confirmed working after disabling hardware offload and rebooting both the pfSense and Ubuntu VMs

8. References

[1] Open Information Security Foundation, "Suricata User Guide," Suricata Documentation. [Online]. Available: https://docs.suricata.io/

[2] Wazuh, Inc., "Wazuh Documentation," Wazuh. [Online]. Available: https://documentation.wazuh.com/

[3] MITRE Corporation, "MITRE ATT&CK," MITRE ATT&CK Framework. [Online]. Available: https://attack.mitre.org/

[4] Center for Internet Security, "CIS Benchmarks," CIS. [Online]. Available: https://www.cisecurity.org/cis-benchmarks

[5] Netgate, "pfSense Documentation," pfSense Docs. [Online]. Available: https://docs.netgate.com/pfsense/en/latest/